

Helios on Claude Pro — Feasibility Analysis & Operating Prompt

Can the Helios v1 build be completed in 4 weeks on a Claude Pro subscription? · Model: Opus 4.8 (unchanged on Pro) · 2026-06-03

Verdict. Yes — realistically completable in 4 weeks on Pro, but it will be *tight* and requires discipline. Honest estimate: **~70–80% chance with the disciplines below; ~40% without them** (scope creep + burning Opus on boilerplate + cramming into single long sessions). Crucially, switching to Pro **does not lower your quality ceiling** — you keep Opus 4.8, so every output is exactly as good. You only trade *throughput* / *headroom*. The constraint is pacing, not capability.

Why Pro is enough for this specific project (the reasoning)

The factor that flips Pro from "no chance" to "doable": **the code already exists in your repo**. You are *copying and adapting* from `DBT_GUIDE.md`, `MCP_ARCHITECTURE.md §9`, and `semantic_layer.yaml` — not generating from a blank page. Adaptation and debugging cost a fraction of the tokens that net-new generation does.

Rough decomposition of the ~90-hour build:

Work type	~Hours	Claude tokens?
GCP/dbt setup, running <code>dbt build</code> / <code>pytest</code> , BigQuery queries, reading errors	~30 hrs	~0 tokens (you run these)
Reading the 5 critical docs + reference lookups	~10 hrs	~0 (you read)
Adapting existing code (dbt models, MCP skeleton, brief)	~30 hrs	moderate (paste spec → "wire/adapt this")
Debugging + hard reasoning (sessionization, decomposition, MCP)	~20 hrs	heavy — reserve Opus here

So only **about half** the project is Claude-bound, and most of that is cheap adaptation. The genuinely token-hungry part — the keystones and debugging — is maybe 15–20 hours of intensive Opus use spread across 4 weeks.

The binding constraint is not "running out of tokens" — it is Pro's rolling ~5-hour usage window plus the weekly cap. (Anthropic does not publish exact token counts and adjusts them; Opus consumes the budget noticeably faster than Sonnet; Max gives roughly 5×/20× the room.) At 20–25 hrs/week **spread across 4–5 days**, the 5-hour resets work *for* you — you rarely sit in one window long enough to stall. You will occasionally hit a cap mid-debug; the fix is to bounce to token-free work (run dbt, write a test, set up the next thing) until it resets.

The 5 disciplines that get you to ~80%

- Hold the LEAN line.** Build v1 only (spine + 1 MCP server + 1 brief + honest eval). The full 7-agent / memory / autonomy design is what would blow any budget. This is the single biggest lever.
- Copy, don't regenerate.** Always paste the exact `DBT_GUIDE` / `MCP_ARCHITECTURE` section and say "adapt this" — never "write me a ...".
- Model split (do this even though Opus is available):** run Claude Code on **Sonnet for boilerplate** (M0–M2 config/staging, `schema.yml`) and switch to **Opus for the keystones** (M3 sessionization, `decompose_change`, M6 MCP wiring, hard debugging). This can roughly 2–3× your effective Pro runway.
- Spread sessions across days**, ~2–3 focused hours each, with `/clear` between milestones. Don't cram a 6-hour block — that is exactly where Pro stalls.
- When you hit a limit, switch to token-free work** (dbt runs, gcloud, writing a test, reading errors). Treat the cap as a pacing signal, not a blocker.

If Pro genuinely throttles you in Week 3 (the MCP + LLM-brief work is the spikiest), that is the moment a **one-month bump to Max** would pay off — you can downgrade right after. But start on Pro; you likely won't need it.

The prompt to paste at the start of every milestone session

Save this; paste it (filling the `<M#>`) at the top of each Claude Code session, and run `/clear` between milestones:

We're building Helios milestone <M#: name> – LEAN v1 scope ONLY (see docs/planning/LEAN_SCOPE.md and HELIOS_EXECUTION_GUIDE.md Part 4). Architecture is FROZEN: do NOT redesign, do NOT propose new docs, do NOT widen scope. I'm on Claude Pro – optimize hard for token efficiency.

Operating rules for this session:

1. SOURCE OF TRUTH: I will attach the exact spec section (e.g. @docs/architecture/DBT_GUIDE.md §X) and the upstream files. COPY/ADAPT that code – never regenerate from scratch, never read a whole large doc (never open HELIOS_PROJECT_BIBLE.md in full). CLAUDE.md is auto-loaded; trust it.
2. PLAN FIRST, in ≤10 lines: the files to create/edit, the test to write first, and the exact commands I'll run. Then STOP and wait for my "go" before writing code.
3. CANONICAL NAMES ONLY (from CLAUDE.md / semantic_layer.yaml). If a name isn't in the attached sources, STOP and ASK – do not guess. A hallucinated name costs me a wasted regeneration.
4. TEST-FIRST on keystones: write the dbt/golden test, then the code. I run it and paste the real output; you fix from the actual error, not from speculation.
5. BE TERSE: output the code + a one-line citation of the spec section. No essays, no restating the architecture back to me, no recap. If a step is mechanical boilerplate I can do without you, say so.
6. When this milestone's exit gate passes, STOP and name the next milestone. Do not roll ahead.

Two add-ons:

- **First message of the whole project**, append: "Start with *HELIOS_EXECUTION_GUIDE.md Part 5 (milestone M0)* only."
- **When you hit a Pro limit mid-task**: stop prompting, switch to running `dbt / pytest / gcloud` or writing the next test yourself, and resume Claude after the window resets.

Estimates are honest approximations, not guarantees — Anthropic does not publish exact Pro token limits and adjusts them over time. The model (Opus 4.8) is identical on Pro and Max; only usage throughput differs.